

02_feature_engineering.ipynb

Feature Engineering for Web Traffic Forecasting

This notebook loads the cleaned and sampled dataset produced in Notebook 01 and creates:

- Lag features
- Rolling mean and std features
- Date-based features (day of week, month)
- Encoded metadata (language, access type, agent)
- Train/validation split

Output: `features.csv` for modeling.

Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load the cleaned dataset

This file is created from Notebook 01 and contains:

- Log-transformed values
- Sampled pages
- Cleaned missing values

```
df = pd.read_csv("/content/drive/MyDrive/CSC480/Final Project/cleaned_sample.csv")
df.head()
```

		Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2015-07-12
0	Phabricator/Project_management_www.mediawiki.o...		1.945910	1.945910	1.609438	1.945910	2.197225	1.945910	1.609438	0.000000	1.098612	...	1.60
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...		5.493061	5.605802	5.736572	5.429346	5.774552	5.743003	5.493061	5.468060	5.497168	...	5.43
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...		1.386294	2.995732	2.995732	3.433987	3.091042	3.218876	2.890372	5.187386	3.713572	...	2.19
3	Érythrée_fr.wikipedia.org_desktop_all-agents		6.511745	6.242223	6.652863	7.060476	6.304449	6.628041	6.320768	6.204558	8.476788	...	5.32
4	Metallica_es.wikipedia.org_all-access_all-agents		7.336286	7.405496	7.441320	7.358831	7.336286	7.363914	7.383368	7.457032	7.560080	...	7.63

5 rows x 554 columns

Identify date columns

All date columns represent daily log-transformed page views.

We will generate lag and rolling features from these.

```
# metadata columns
meta_cols = ["Page", "language", "access", "agent"]

# date columns
date_cols = [c for c in df.columns if c not in meta_cols]
```

```
print("Number of dates:", len(date_cols))
print("Total rows:", df.shape[0])
```

Number of dates: 550
Total rows: 500

Create Lag Features

We generate lag features of:

- 7 days
- 14 days
- 30 days

These capture short-term temporal dependencies.

```
df_lag = df.copy()

LAGS = [7, 14, 30]

for lag in LAGS:
    shifted = df_lag[date_cols].shift(lag, axis=1)
    shifted.columns = [f"{c}_lag{lag}" for c in date_cols]
    df_lag = pd.concat([df_lag, shifted], axis=1)

df_lag.head()
```

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2015-07-22
0	Phabricator/Project_management_www.mediawiki.o...	1.945910	1.945910	1.609438	1.945910	2.197225	1.945910	1.609438	0.000000	1.098612	...	2.0
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...	5.493061	5.605802	5.736572	5.429346	5.774552	5.743003	5.493061	5.468060	5.497168	...	6.0
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...	1.386294	2.995732	2.995732	3.433987	3.091042	3.218876	2.890372	5.187386	3.713572	...	1.6
3	Érythrée_fr.wikipedia.org_desktop_all-agents	6.511745	6.242223	6.652863	7.060476	6.304449	6.628041	6.320768	6.204558	8.476788	...	6.1
4	Metallica_es.wikipedia.org_all-access_all-agents	7.336286	7.405496	7.441320	7.358831	7.336286	7.363914	7.383368	7.457032	7.560080	...	8.4

5 rows x 2204 columns

Rolling Features

Rolling mean and std provide smoothed representations of trends. Window sizes: 7-day rolling.

```
WINDOW = 7

# rolling mean across all date columns
roll_mean = df_lag[date_cols].rolling(WINDOW, axis=1).mean()
roll_mean.columns = [f"{c}_roll_mean" for c in date_cols]

# rolling std across all date columns
roll_std = df_lag[date_cols].rolling(WINDOW, axis=1).std()
roll_std.columns = [f"{c}_roll_std" for c in date_cols]

# add them to the dataframe
df_lag = pd.concat([df_lag, roll_mean, roll_std], axis=1)

df_lag.head()
```

```
/tmp/ipython-input-345540429.py:4: FutureWarning: Support for axis=1 in DataFrame.rolling is deprecated and will be removed in a future version of pandas.
roll_mean = df_lag[date_cols].rolling(WINDOW, axis=1).mean()
/tmp/ipython-input-345540429.py:8: FutureWarning: Support for axis=1 in DataFrame.rolling is deprecated and will be removed in a future version of pandas.
roll_std = df_lag[date_cols].rolling(WINDOW, axis=1).std()
```

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	22_1
0	Phabricator/Project_management_www.mediawiki.o...	1.945910	1.945910	1.609438	1.945910	2.197225	1.945910	1.609438	0.000000	1.098612	...	
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...	5.493061	5.605802	5.736572	5.429346	5.774552	5.743003	5.493061	5.468060	5.497168	...	
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...	1.386294	2.995732	2.995732	3.433987	3.091042	3.218876	2.890372	5.187386	3.713572	...	
3	Érythrée_fr.wikipedia.org_desktop_all-agents	6.511745	6.242223	6.652863	7.060476	6.304449	6.628041	6.320768	6.204558	8.476788	...	
4	Metallica_es.wikipedia.org_all-access_all-agents	7.336286	7.405496	7.441320	7.358831	7.336286	7.363914	7.383368	7.457032	7.560080	...	

5 rows x 3304 columns

Encode the metadata

We convert:

- language
- access
- agent

into numeric form using label encoding.

```
le_lang = LabelEncoder()
le_acc = LabelEncoder()
le_agent = LabelEncoder()

df_lag["lang_enc"] = le_lang.fit_transform(df_lag["language"])
df_lag["access_enc"] = le_acc.fit_transform(df_lag["access"])
df_lag["agent_enc"] = le_agent.fit_transform(df_lag["agent"])

df_lag.head()
```

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	25_1
0	Phabricator/Project_management_www.mediawiki.o...	1.945910	1.945910	1.609438	1.945910	2.197225	1.945910	1.609438	0.000000	1.098612	...	
1	Now_You_See_Me_es.wikipedia.org_desktop_all-ag...	5.493061	5.605802	5.736572	5.429346	5.774552	5.743003	5.493061	5.468060	5.497168	...	
2	Zürich_Hackathon_2014_www.mediawiki.org_all-ac...	1.386294	2.995732	2.995732	3.433987	3.091042	3.218876	2.890372	5.187386	3.713572	...	
3	Érythrée_fr.wikipedia.org_desktop_all-agents	6.511745	6.242223	6.652863	7.060476	6.304449	6.628041	6.320768	6.204558	8.476788	...	
4	Metallica_es.wikipedia.org_all-access_all-agents	7.336286	7.405496	7.441320	7.358831	7.336286	7.363914	7.383368	7.457032	7.560080	...	

5 rows x 3307 columns

Create Train/Validation Split

We forecast the next **60 days**.

- Training data: all except last 60 dates
- Validation data: last 60 dates

```
FORECAST_HORIZON = 60

train_dates = date_cols[:-FORECAST_HORIZON]
val_dates = date_cols[-FORECAST_HORIZON:]

print("Training days:", len(train_dates))
print("Validation days:", len(val_dates))
```

```
Training days: 490
Validation days: 60
```

Prepare train and validation datasets

We reshape:

- X = features from lag + metadata
- y = actual future values

```
# Features will be the last valid lag feature of each row
feature_cols = [c for c in df_lag.columns if "lag" in c or "roll" in c] + ["lang_enc", "access_enc", "agent_enc"]

X = df_lag[feature_cols]
y = df_lag[val_dates] # next 60 days

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (500, 2753)
y shape: (500, 60)
```

✓ Save the engineered features

These files will be used in:

- lightgbm.ipynb
- lstm.ipynb
- ensemble.ipynb

```
save_dir = "/content/drive/MyDrive/CSC480/Final Project/"
df_lag.to_csv(save_dir + "features.csv", index=False)
y.to_csv(save_dir + "labels.csv", index=False)

print("Saved features and labels.")
```

```
Saved features and labels.
```